

Restoring Our Kelp Forests: A Multi-Trophic Coastal Ecosystem Simulator

BRYAN AGAMU, Kennesaw State University, CS 4632: Modeling and Simulation, USA

Wild Kelp forests have been observed to abruptly flip from a lush garden into low diversity barren landscapes typically referred to as “urchin deserts”. This happens when their key grazer, the sea urchin, loses its keystone predator the sea otter. This paper describes an agent-based simulation of a three-level trophic system (kelp, sea urchins, sea otters) on a two-dimensional spatial grid. The simulation will be implemented in Python and will track seven quantitative metrics across seeded Monte Carlo runs.

1 Project Overview

Project Title: Understanding Kelp Forest Collapse & Recovery: A Coastal Ecosystem Simulator.

Domain: Marine ecology and population dynamics.

1.1 Problem Statement

Kelp forests can exist in two different states: a lush healthy canopy that supports high biodiversity, or a reduced “urchin barren” where sea urchins have stripped all of the rocks bare. My simulation is designed to answer:

- At what sea-otter mortality rate does the system collapse into a barren?
- Is collapse reversible by simply restoring otters, or does recovery require more work than the disturbance that triggered collapse?
- How does the spatial clustering of urchins change outcomes, and what is the most effective method of recovery?

1.2 Scope

Will simulate: Three trophic levels as discrete agents/resources on a grid, spatial foraging, movement, reproduction, and death, stochastic disturbances (storms, disease).

Will not simulate: The full marine food cycle (plankton, fish, invertebrate diversity). Ocean chemistry, currents, or fluid dynamics, genetic or evolutionary adaptation, and real geographic coastlines (an abstract rectangular grid is used).

2 System Description

2.1 System Components

Table 1. Core entities, their state, and behaviors.

Entity	Key properties	Behaviors
Kelp (resource)	biomass, growth rate, max density	They regrow or are consumed by urchins
Sea urchin (eats Kelp)	position, energy, age, reproduction threshold	They graze, move, reproduce, flee, die / be eaten
Sea otter (eats Urchin)	position, energy, hunger, age	They hunt urchins, raft, reproduce, starve and die
Grid cell (rocks that grow Kelp)	substrate, temperature, disturbance state	It hosts kelp and propagates storm events

2.2 System Dynamics

Components interact across all three trophic levels every time step:

- **Top-down control:** otters eat urchins, suppressing the herbivore population. Less urchins means more kelp growth
- **Bottom-up limitation:** local kelp density bounds urchin energy intake and reproduction. Less kelp density means urchins can intake and reproduce more effectively
- **Collapse cascade:** removing otters lets urchins explode, kelp is grazed away, a barren forms, urchins then starve, the system settles into a low-biomass “urchin desert” stable state.
- **Spatial coupling:** local depletion forces movement, producing emergent urchin grazing fronts, otter rafts, and kelp refuges that a non-spatial model would not capture.
- **Stochastic disturbance:** random storms and disease arrive as events that kill predators and hence, reset kelp patches. They may locally and temporarily rescue or doom populations.

2.3 Core Models and Algorithms

(1) *Lotka–Volterra dynamics with carrying capacity.* The classical predator–prey system [4, 7] provides the baseline rate structure. In continuous form with logistic prey growth,

$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K}\right) - f(N)P, \quad (1)$$

$$\frac{dP}{dt} = e f(N)P - mP, \quad (2)$$

where N is prey (urchin) density, P is predator (otter) density, r the prey growth rate, K the carrying capacity, e a conversion efficiency, and m the predator mortality. I reduced these rates into per-step probabilities that govern reproduction and predation for each agent. *Equations from:* [4, 7].

(2) *Holling Type II functional response.* Predation saturates at high prey density because predators need handling time per kill [3]:

$$f(N) = \frac{aN}{1 + ahN}, \quad (3)$$

with attack rate a and handling time h . This sets each otter’s per-step probability of a successful hunt as a function of nearby urchin density. *Equations from:* [3].

(3) *Logistic resource regeneration.* Each cell’s kelp biomass B regrows toward a light-limited maximum $B_{\max}$:

$$B_{t+1} = B_t + g B_t \left(1 - \frac{B_t}{B_{\max}}\right) - C_t, \quad (4)$$

where g is the growth rate and C_t the biomass grazed that step. This is the bottom-up bound coupling resources to consumer dynamics. *Equations from:* [4].

(4) *Stochastic event model.* Reproduction, death, and predation outcomes are Bernoulli draws each tick; environmental disturbances (storms, disease) arrive as Poisson events with rate λ , $P(k \text{ events}) = \lambda^k e^{-\lambda} / k!$. This supplies the simulation’s required randomness and lets us study variance and extinction frequency across different seeds. *Equations from:* [5].

2.4 Assumptions

The grid is closed, so agents cannot migrate across its boundaries. Kelp is modeled as continuous blob of biomass per cell rather than individual plants. Each step is a fixed biological interval, such as one week. Agents of the same type share the same parameters within a species, which is not true in nature. Otters eat only urchins and urchins eat only kelp, so each level has a single prey. Disturbances are local and independent of one another.

3 Implementation Approach

Programming Language. Python, chosen for rapid prototyping, readable logic and useful libraries.

Environment / Libraries. **NumPy** for grid state and vectorized rate computation, **Mesa** for agent-based scheduling, **Matplotlib** for generating population time series graphs and grid, & snapshots **pandas** for metrics logging and analysis, **pytest** for testing, **Git/GitHub** for version control.

Simulation Type. Agent-based, discrete-time simulation, with continuous-equation rate models setting individual agent decisions.

Data Collection Plan. The model logs seven metrics: (1) per-species population time series (counts and biomass); (2) kelp coverage percentage over time; (3) the tipping-point threshold (otter mortality at collapse); (4) the hysteresis gap between collapse and recovery thresholds; (5) a spatial clustering index (Moran's I) for urchins; (6) time-to-recovery under each intervention; and (7) ecosystem stability/variance and extinction frequency across seeded runs.

4 Literature Review

Lotka (1925) [4]. Establishes the coupled differential equations of interacting populations. It provides our baseline predator–prey rate structure. I discretize the equations into per-agent, per-step probabilities and add an explicit carrying-capacity term.

Volterra (1926) [7]. Independently derives swaying predator–prey dynamics and the conservation laws governing them. It predicts the population cycles our model should reproduce before collapse dynamics are introduced.

Holling (1959) [3]. Introduces functional response types and the handling-time mechanism behind predator saturation. It governs otter predation rate as a function of urchin density. The Type II curve is converted to a per-step hunt probability bounded by handling time.

Estes & Palmisano (1974) [1]. The seminal empirical demonstration that sea otters are a keystone predator whose removal triggers urchin barrens. It justifies the otter as the control variable for collapse, and I use it to parameterize the collapse cascade and to ground-truth the qualitative barren-formation behavior.

Scheffer et al. (2001) [6]. A synthesis of alternative stable states, catastrophic shifts, and hysteresis in ecosystems. It is the theoretical backbone for our tipping-point and hysteresis questions, and it informs the experimental design that sweeps mortality up (to find collapse) and back down (to find recovery), the gap between which quantifies hysteresis.

Filbee-Dexter & Scheibling (2014) [2]. A focused review treating urchin barrens explicitly as an alternative stable state of collapsed kelp ecosystems, including spatial and feedback mechanisms. It directly matches our domain and validates the two-state framing, and it guides parameter ranges and the spatial mechanisms (grazing fronts, refugia) we implement.

Railsback & Grimm (2019) [5]. A practical text on agent-based and individual-based modeling, scheduling, and the ODD protocol. It provides architectural guidance for the simulation loop and scheduler: our class structure and per-tick activity flow follow its design patterns, and results documentation will follow the ODD protocol.

5 System Architecture (UML)

5.1 Class Diagram

Figure 1 shows the static architecture. An abstract Agent base class is specialized by Urchin and Otter via inheritance. Kelp is a resource owned by each GridCell; Grid is composed of cells; and Simulation composes the Grid, Scheduler, DisturbanceModel, DataCollector, and Params. Dashed dependencies capture the trophic interactions (otters prey on urchins; urchins graze kelp) and the observer relationship of the data collector.

5.2 Activity Diagram

Figure 2 shows the per-tick control flow of the main simulation loop. First the kelp regrows. Then the agents are activated in a shuffled order to sense and act (hunt, flee, move, graze) under their respective models. Next the model resolves reproduction and death, and fires stochastic disturbance events. Finally it collects metrics and advances the tick. The loop repeats until the step limit or extinction is reached.

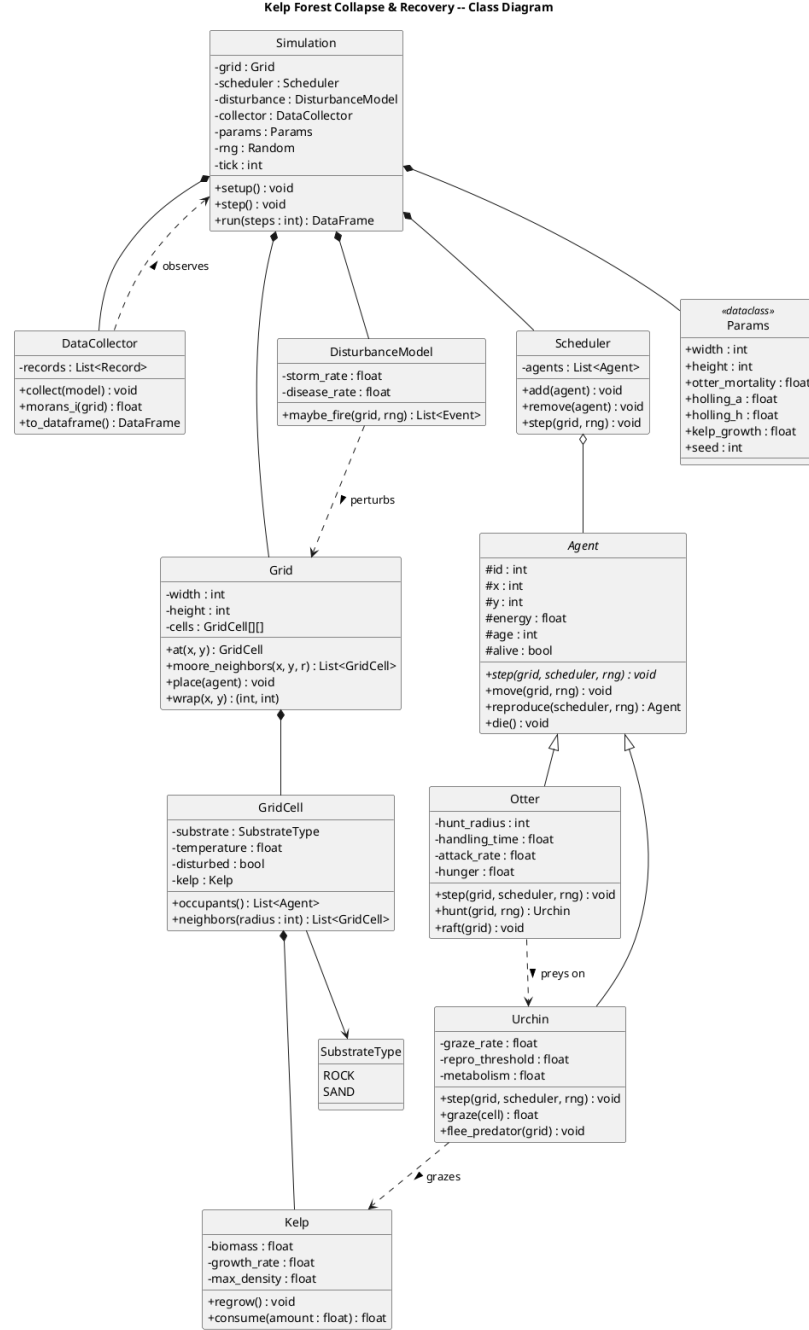


Fig. 1. Class diagram of the simulation architecture, showing inheritance (Agent→Urchin/Otter) and composition (Simulation, Grid, GridCell).

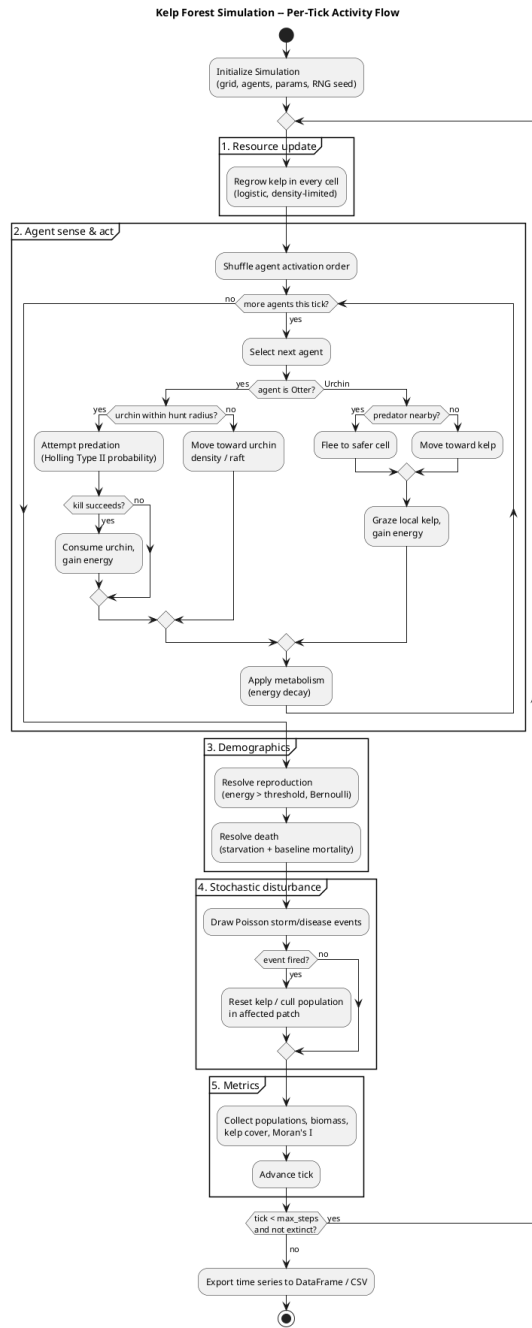


Fig. 2. Activity diagram of the per-tick simulation loop.

6 Repository

All code, diagrams, and this document are version-controlled at the project GitHub repository (<https://github.com/jxc2000-b/msim-proj/blob/main/README.md>). The initial commit includes a `README.md`, the Python project structure, a `Python/` $\LaTeX$ `.gitignore`, the UML sources, and this proposal.

7 Note on AI Use

An AI assistant was used as a tool during this project to help scaffold and debug the simulation code, draft and format this $\LaTeX$ document from the ACM template, plus help generate and clean the PlantUML UML diagrams. All design decisions, model choices, parameters, and the research questions are my own, and I have reviewed and verified the resulting code, text, and citations.

References

- [1] James A. Estes and John F. Palmisano. 1974. Sea Otters: Their Role in Structuring Nearshore Communities. *Science* 185, 4156 (1974), 1058–1060. doi:10.1126/science.185.4156.1058
- [2] Karen Filbee-Dexter and Robert E. Scheibling. 2014. Sea Urchin Barrens as Alternative Stable States of Collapsed Kelp Ecosystems. *Marine Ecology Progress Series* 495 (2014), 1–25. doi:10.3354/meps10573
- [3] C. S. Holling. 1959. Some Characteristics of Simple Types of Predation and Parasitism. *The Canadian Entomologist* 91, 7 (1959), 385–398. doi:10.4039/Ent91385-7
- [4] Alfred J. Lotka. 1925. *Elements of Physical Biology*. Williams & Wilkins, Baltimore.
- [5] Steven F. Railsback and Volker Grimm. 2019. *Agent-Based and Individual-Based Modeling: A Practical Introduction* (2 ed.). Princeton University Press, Princeton, NJ.
- [6] Marten Scheffer, Steve Carpenter, Jonathan A. Foley, Carl Folke, and Brian Walker. 2001. Catastrophic Shifts in Ecosystems. *Nature* 413, 6856 (2001), 591–596. doi:10.1038/35098000
- [7] Vito Volterra. 1926. Fluctuations in the Abundance of a Species Considered Mathematically. *Nature* 118, 2972 (1926), 558–560. doi:10.1038/118558a0